

ECA1414- EMBEDDED SYSTEM

CO2-AT-01

GITHUB LINK: <https://github.com/Pranavvanaraju/ECA1414-Embedded-System>

NAME: V Pranav

REG.NO:192311029

DATE:06/08/2026

FACULTY: DR G P BHARATHI

CODE:

```
#include <stdio.h>

/* Structure for sensor interface */
typedef struct
{
    char name[20];

    /* Function pointers */
    void (*init)(void);
    float (*read)(void);

} Sensor;

/* Temperature sensor functions */
void Temperature_Init(void)
{
    printf("Temperature Sensor Initialized\n");
}

float Temperature_Read(void)
{
    return 28.5;
}

/* Humidity sensor functions */
void Humidity_Init(void)
{
    printf("Humidity Sensor Initialized\n");
}
```

```
float Humidity_Read(void)
{
    return 65.0;
}
```

```
int main(void)
{
    /* Create temperature sensor */
    Sensor temperatureSensor =
    {
        "Temperature",
        Temperature_Init,
        Temperature_Read
    };

    /* Create humidity sensor */
    Sensor humiditySensor =
    {
        "Humidity",
        Humidity_Init,
        Humidity_Read
    };

    /* Initialize sensors using function pointers */
    temperatureSensor.init();
    humiditySensor.init();

    /* Read sensor values using function pointers */
    printf("%s: %.2f Degree Celsius\n",
```

```

        temperatureSensor.name,
        temperatureSensor.read());

    printf("%s: %.2f Percent\n",
        humiditySensor.name,
        humiditySensor.read());

    return 0;
}

```

OUTPUT:

The screenshot shows the Programiz C Online Compiler interface. The code in the editor is as follows:

```

1 #include <stdio.h>
2
3 /* Structure for sensor interface */
4 typedef struct
5 {
6     char name[20];
7
8     /* Function pointers */
9     void (*init)(void);
10    float (*read)(void);
11
12 } Sensor;
13
14 /* Temperature sensor functions */
15 void Temperature_Init(void)
16 {
17     printf("Temperature Sensor Initialized\n");
18 }
19
20 float Temperature_Read(void)
21 {
22     return 28.5;
23 }
24
25 /* Humidity sensor functions */
26 void Humidity_Init(void)

```

The output window shows the following results:

```

^
Temperature Sensor Initialized
Humidity Sensor Initialized
Temperature: 28.50 Degree Celsius
Humidity: 65.00 Percent

=== Code Execution Successful ===

```

CODE:

```

#include <stdio.h>

/* Hardware Abstraction Layer (HAL) declarations */

void HAL_Init(void);

float HAL_ReadTemperature(void);

void HAL_DisplayTemperature(float value);

```

```
/* HAL function definitions */
```

```
/* Initialize hardware */
```

```
void HAL_Init(void)
{
    printf("Hardware Initialized\n");
}
```

```
/* Read temperature from hardware */
```

```
float HAL_ReadTemperature(void)
{
    return 30.5;
}
```

```
/* Display temperature */
```

```
void HAL_DisplayTemperature(float value)
{
    printf("Temperature: %.2f Degree Celsius\n", value);
}
```

```
/* Main program */
```

```
int main(void)
```

```
{
    float temperature;
```

```
    /* Initialize hardware */
```

```
    HAL_Init();
```

```
    /* Read temperature using HAL */
```

```
    temperature = HAL_ReadTemperature();
```

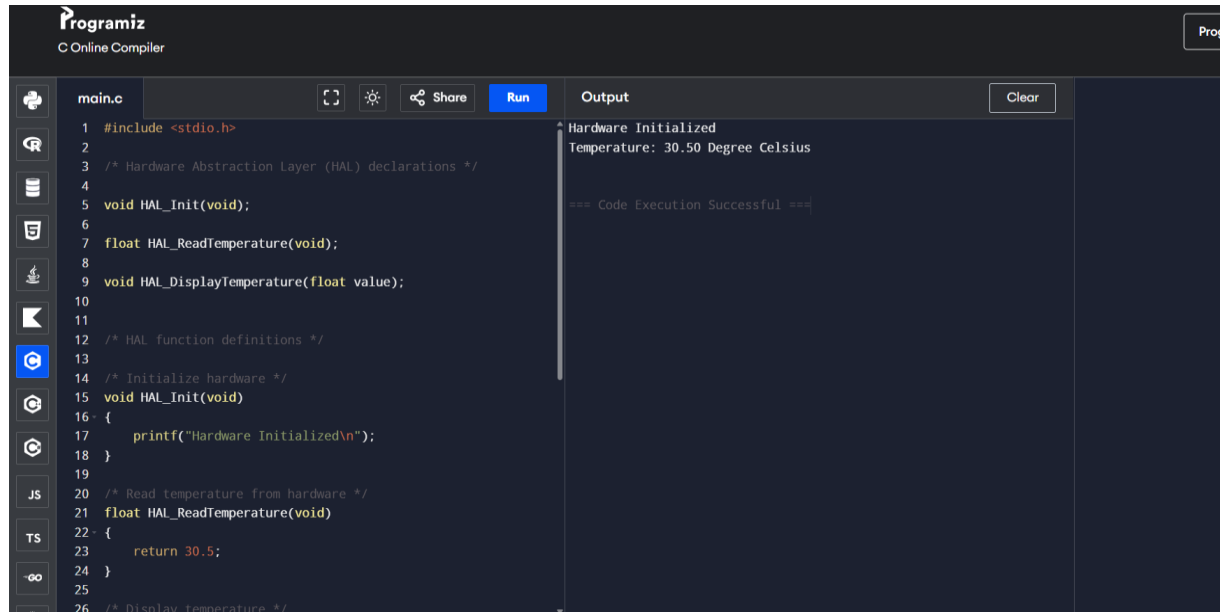
```
    /* Display temperature using HAL */
```

```
    HAL_DisplayTemperature(temperature);
```

```
    return 0;
```

```
}
```

OUTPUT:



The screenshot displays the Programiz C Online Compiler interface. On the left, a sidebar contains icons for various programming languages, with C selected. The main editor area shows a C program named `main.c` with the following code:

```
1 #include <stdio.h>
2
3 /* Hardware Abstraction Layer (HAL) declarations */
4
5 void HAL_Init(void);
6
7 float HAL_ReadTemperature(void);
8
9 void HAL_DisplayTemperature(float value);
10
11
12 /* HAL function definitions */
13
14 /* Initialize hardware */
15 void HAL_Init(void)
16 {
17     printf("Hardware Initialized\n");
18 }
19
20 /* Read temperature from hardware */
21 float HAL_ReadTemperature(void)
22 {
23     return 30.5;
24 }
25
26 /* Display temperature */
```

On the right, the 'Output' panel shows the results of the program execution:

```
Hardware Initialized
Temperature: 30.50 Degree Celsius

=== Code Execution Successful ===
```

The 'Run' button is highlighted in blue, and the 'Clear' button is visible in the top right of the output panel.